

Image Hosting Service

Delivery roadmap and progress checklist

Current checkpoint: authenticated upload and owner-only content retrieval are working end to end.

Progress at a glance

COMPLETED

NEXT

PLANNED

1. Upload + content	2. Owner library	3. Delete + visibility
4. Async processing	5. AI tagging	6. Thumbnails
7. Public gallery + search	8. Sharing + hardening	9. Cloud + frontend

RESUME HERE

Finish `ImageRepository.countByOwnerId()`, create reusable `PageResponse<T>`, and expose `GET /api/v1/images/mine`.

1. Working foundation

These capabilities have been implemented and manually verified with Postman. Automated coverage is still required before merge.

Authenticated upload

- ✓ Accept authenticated multipart uploads at **POST /api/v1/images**.
- ✓ Accept JPEG and PNG files up to 10 MB.
- ✓ Sanitize filenames and reject empty or oversized uploads.
- ✓ Decode actual image content and record width and height.
- ✓ Upload repeatable request bytes to private S3-compatible storage.
- ✓ Store object key and metadata in PostgreSQL.

Private content retrieval

- ✓ Find image metadata by primary key.
- ✓ Allow the owner to retrieve a private image.
- ✓ Return the same 404 response for missing and inaccessible private images.
- ✓ Stream private object content through the backend.
- ✓ Set Content-Type, Content-Length, inline disposition, and nosniff headers.
- ✓ Manually verify a newly uploaded image renders through the content endpoint.

Deferred stabilization

- ☐ Add validator unit tests.
- ☐ Add upload service and controller tests.
- ☐ Add owner, non-owner, missing-image, and streaming tests.

2. Owner library and image management

Build the authenticated user's library before exposing public behavior. Each endpoint must enforce ownership and return predictable errors.

Owned-image listing - next

- ☒ Add `countByOwnerId()` for pagination metadata.
- ☐ Create reusable `PageResponse<T>`.
- ☐ Implement service pagination with page and size validation.
- ☐ Expose **GET** `/api/v1/images/mine?page=0&size=20`.
- ☐ Return public and private images newest first.
- ☐ Add stable ordering and owner-listing tests.

Image metadata

- ☐ Expose image metadata by ID.
- ☐ Allow public metadata later and owner-only private metadata.
- ☐ Avoid exposing storage keys or provider details.

Owner-only deletion

- ☐ Expose **DELETE** `/api/v1/images/{imageId}`.
- ☐ Verify ownership before deleting anything.
- ☐ Delete the original object and thumbnail when present.
- ☐ Delete the database record safely.
- ☐ Define recovery behavior for partial storage/database failures.
- ☐ Prevent and clean up orphaned storage objects.

Database performance

- ☐ Add a Flyway migration for the final owner-listing index.
- ☐ Align index ordering with `owner_id`, `created_at` DESC, and `id` DESC.
- ☐ Use EXPLAIN ANALYZE after representative data exists.

3. Visibility, processing, and thumbnails

Make visibility explicit, then add asynchronous processing without slowing the upload response.

Public and private visibility

- ☐ Accept an explicit visibility choice during upload.
- ☐ Expose an owner-only visibility update endpoint.
- ☐ Allow anonymous access to public content and metadata.
- ☐ Keep private content available only to its owner or a valid share link.
- ☐ Exclude private images from galleries and search.
- ☐ Return indistinguishable 404 responses for missing and inaccessible private images.
- ☐ Add public/private authorization tests.

Asynchronous processing lifecycle

- ☐ Keep upload response independent from AI and thumbnail work.
- ☐ Support PENDING, PROCESSING, COMPLETED, and FAILED states.
- ☐ Record processing start, completion, duration, and failure details.
- ☐ Define retry behavior and idempotency.
- ☐ Ensure processing failures never delete a successful original upload.

100 x 100 thumbnails

- ☐ Generate a square 100 x 100 thumbnail with a documented crop strategy.
- ☐ Store thumbnails in private object storage.
- ☐ Save the thumbnail storage key in PostgreSQL.
- ☐ Expose a secured thumbnail content endpoint.
- ☐ Use thumbnails in galleries, search, and the user library.
- ☐ Test dimensions, formats, access rules, and failure handling.

Validation hardening

- ☐ Verify declared MIME type matches decoded image bytes.
- ☐ Decide whether to support WebP or remove it from schema and documentation.
- ☐ Reject unreasonable dimensions and decompression-bomb inputs.
- ☐ Return deliberate 400, 413, 415, and storage-failure responses.

4. AI, discovery, sharing, and delivery

Complete the product experience, operational safeguards, and bonus capabilities only after the core image lifecycle is stable.

AI image tagging

- ☐ Send original images to a vision-capable AI model asynchronously.
- ☐ Extract visible objects, descriptive tags, and up to three prominent colors.
- ☐ Require and validate a structured AI response.
- ☐ Store AI metadata in PostgreSQL JSONB.
- ☐ Record AI timing, failures, and retry outcomes.
- ☐ Test successful, malformed, timed-out, and failed AI responses.

Public gallery and search

- ☐ Return the latest 50 public images.
- ☐ Search public images by objects, tags, colors, and free text.
- ☐ Paginate search results with stable ordering.
- ☐ Add only query-proven B-tree, GIN, full-text, or trigram indexes.
- ☐ Expose public user galleries without private metadata leakage.

Private sharing

- ☐ Generate cryptographically secure share tokens.
- ☐ Allow owners to choose expiration times.
- ☐ Serve private content through valid, unexpired tokens.
- ☐ Allow owners to revoke links.
- ☐ Test expiry, revocation, ownership, and token secrecy.

Operations, documentation, and frontend

- ☐ Add structured logs, request correlation, and access-denied audit events.
- ☐ Measure upload, storage, thumbnail, AI, search, and failure metrics.
- ☐ Keep OpenAPI as a living contract that follows verified design decisions.
- ☐ Complete comprehensive unit, integration, security, and end-to-end tests.
- ☐ Verify Docker, CI tests, GHCR publishing, and cloud configuration.
- ☐ Use AI to design and build the upload flow, owner library, public gallery, search, detail, and sharing frontend.
- ☐ Connect frontend requests to sessions, CSRF protection, error states, loading states, and request correlation.